High-Level Design (HLD)

Project: FinFlow Loan Management System

---

1. System Overview

1.1 Purpose

FinFlow is a web-based loan onboarding and management platform that enables applicants to apply for loans digitally and allows administrators to review, verify, and process applications efficiently.

1.2 Scope

The system supports:

- User registration and authentication
- Loan application submission via guided workflow
- Document upload and verification
- Application tracking
- Admin review and decision-making
- Reporting and user management

1.3 Out of Scope

- Third-party credit scoring integration
- Payment disbursement systems

- Advanced analytics and AI-based recommendations

---

2. Architecture Overview

2.1 Architecture Style

- Microservices Architecture
- API Gateway Pattern
- Combination of synchronous (REST) and secure communication

2.2 High-Level Flow

Client (Angular) → API Gateway → Microservices → Databases

2.3 Core Components

- Frontend (Angular Application)
- API Gateway (Spring Cloud Gateway)
- Backend Microservices (Spring Boot)
- Databases (MySQL/PostgreSQL)

---

3. Technology Stack

Layer	Technology
Frontend	Angular
API Gateway	Spring Cloud Gateway
Backend Services	Spring Boot (Java 17)
Security	Spring Security (JWT)
Database	MySQL / PostgreSQL
ORM	JPA / Hibernate
Build Tool	Maven
Testing Backend	JUnit, Mockito
Testing Frontend	Jasmine, Karma

---

4. Microservices Design

4.1 Auth Service

Responsibilities:

- User registration and authentication
- JWT token generation and validation
- Role-based access management

4.2 Application Service

Responsibilities:

- Loan application lifecycle management
- Draft creation and submission
- Status tracking

4.3 Document Service

Responsibilities:

- Document upload and storage
- Document verification workflow

4.4 Admin Service

Responsibilities:

- Application review and decision-making
- Report generation
- User management

---

5. API Gateway Design

Responsibilities:

- Central routing of all API requests
- Authentication validation (JWT forwarding)
- Request filtering and logging

Routes:

- /gateway/auth/**
- /gateway/applications/**
- /gateway/documents/**
- /gateway/admin/**

---

6. UI Architecture (Angular)

Modules:

- CoreModule (interceptors, layout, error handling)
- SharedModule (reusable components)
- AuthModule (login, signup)
- ApplicantModule (dashboard, apply, documents, status)
- AdminModule (dashboard, review, users)

- ReportsModule (analytics and charts)

Routing Structure:

- /auth/login
- /auth/signup
- /applicant/dashboard
- /applicant/apply
- /applicant/documents
- /applicant/status/:id
- /admin/dashboard
- /admin/review/:id
- /admin/reports
- /admin/users

---

7. Data Flow

Loan Application Flow:

1. Applicant logs in and creates application
2. Application saved as draft

3. Applicant submits application
4. Documents uploaded
5. Admin verifies documents
6. Application reviewed by admin
7. Decision taken (Approved/Rejected)
8. Status updated and visible to applicant

---

8. Database Design (High-Level)

Approach:

- Each microservice owns its database/schema
- No direct database sharing
- Communication via APIs

Key Entities:

User

- id
- name
- email

- password
- role

LoanApplication

- id
- user_id
- status
- loan_amount
- created_at

Document

- id
- application_id
- document_type
- status

Decision

- id
- application_id
- decision
- remarks

Report

- id
- type
- generated_at

---

9. Security Design

Authentication:

- JWT-based authentication
- Token generated by Auth Service

Authorization:

- Role-based access control (Applicant, Admin)
- Implemented using Spring Security annotations

Frontend Security:

- Angular route guards
- HTTP interceptor for token injection

---

10. Application Status Lifecycle

Draft → Submitted → Docs Pending → Docs Verified → Under Review → Approved / Rejected → Closed

---

11. Non-Functional Requirements

11.1 Scalability

- Microservices enable independent scaling
- Stateless services for horizontal scaling

11.2 Performance

- Optimized database queries
- Pagination for large datasets

11.3 Availability

- Independent deployment of services
- Fault isolation between services

11.4 Security

- JWT authentication
- HTTPS communication
- Role-based authorization

11.5 Reliability

- Input validation at all layers
- Exception handling mechanisms

11.6 Observability

- Centralized logging
- Monitoring support (future enhancement)

---

12. Deployment Architecture

Deployment Flow:

- Angular app deployed on web server
- Microservices deployed as independent services
- API Gateway exposed publicly
- Databases hosted per service

Containerization (Optional Enhancement):

- Docker containers for each service
- Network-based communication
- Volume-based persistence

---

13. Integration Points

- Frontend ↔ API Gateway
- Gateway ↔ Microservices
- Microservices ↔ Databases

(No external third-party integrations in current scope)

---

14. Assumptions

- All users have valid internet access
- JWT tokens are securely stored on client side
- System operates in a single region

---

15. Constraints

- No distributed transactions across services
- Limited real-time event processing
- No API rate limiting implemented

---

16. Design Decisions

Decision	Reason
Microservices Architecture	Scalability and modularity
API Gateway	Centralized routing and security
JWT Authentication	Stateless and scalable security
Separate Databases	Service independence
Angular Frontend	Modular UI development

---

17. Trade-offs

Choice	Trade-off
Microservices	Increased complexity
JWT	Token management overhead
Separate DB per service	Data duplication risk
Gateway routing	Slight latency increase

---

18. Future Enhancements

- Service Discovery (Eureka)
- Circuit Breaker (Resilience4j)
- Centralized Logging (ELK Stack)
- Monitoring (Prometheus, Grafana)

- Kubernetes-based deployment
- Caching (Redis)

---

19. Summary

The FinFlow system is designed using a microservices architecture with a centralized API Gateway and JWT-based security. The system ensures modularity, scalability, and maintainability by separating concerns across services such as authentication, application processing, document management, and administration. The architecture supports a complete digital loan processing lifecycle with secure and efficient workflows.